

4.5、句型的分析

句型分析：识别一个**符号串**是否为某文法的**句型**，是构造某个**推导**的过程。

在语言的编译实现中，把完成**句型分析**的程序称为**分析程序**或**识别程序**。分析算法又称**识别算法**。

从左到右的识别算法，即总是从左到右地识别输入符号串，首先识别符号串中的最左符号，进而依次识别右边的一个符号，直到分析结束。

句型的分析算法分类

自上而下分析法

从文法的**开始符号**出发，反复使用文法的产生式，寻找与输入符号串匹配的推导。

语法树的构造过程：从文法符号开始，将它做为语法树的根，向下逐步建立**语法树**，使语法树的**所有叶子正好是输入符号串**

例：自上而下的语法分析

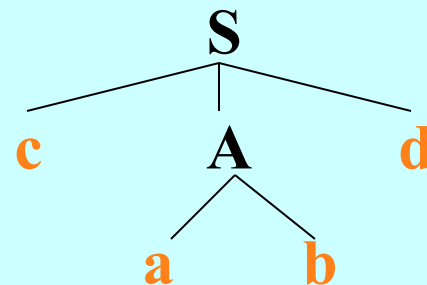
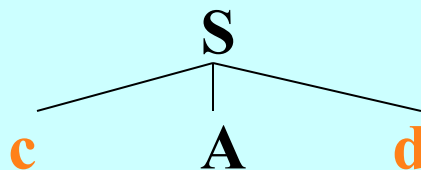
文法G: $S \rightarrow cAd$

$A \rightarrow ab$

$A \rightarrow a$

识别输入串 $w=cabd$ 是否为该文法的句子

S



推导过程: $S \Rightarrow cAd$ $cAd \Rightarrow c\underline{a}bd$

显然，这种自上而下分析是一个不断试探的过程；也即，在分析过程中，如果出现多个产生式(即候选式)可供选择，则逐一试探每一候选式进行匹配，每当一次试探失败，就选取下一候选式再进行试探；此时，必须回溯到这一次试探的初始现场，包括注销已生长的子树及将匹配指针调回到失败前的状态。这种带回溯的自上而下分析方法实际上是一种穷举的试探方法，其分析效率极低，在实用的编译程序中很少使用。



句型的分析算法分类

自下而上分析法

从输入符号串开始，逐步进行归约，直至归约到文法的开始符号。

语法树的构造过程：从输入符号串开始，以它做为语法树的叶子，自底向上地构造语法树，直到根。

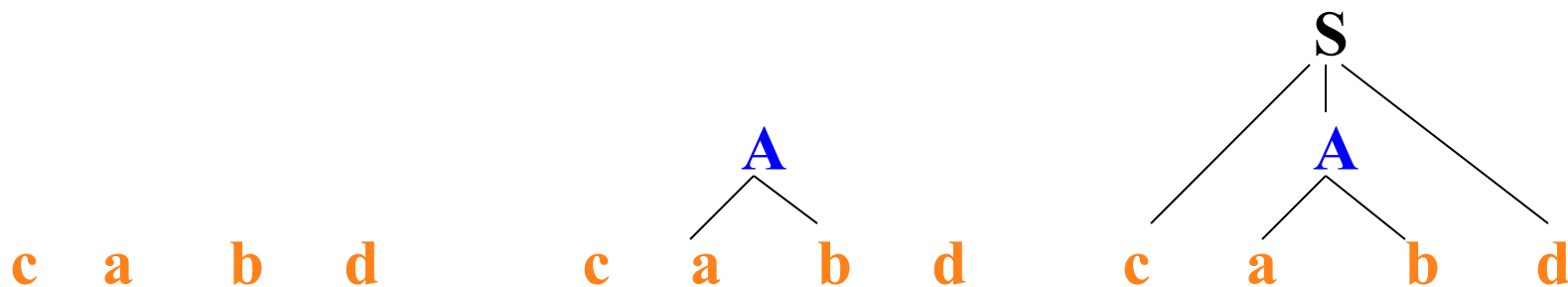
例：自下而上的语法分析

文法G: $S \rightarrow cAd$

$A \rightarrow ab$

$A \rightarrow a$

识别输入串 $w=cabd$ 是否该文法的句子



规约过程构造的推导: $cAd \Rightarrow \underline{cabd}$ $S \Rightarrow cAd$

所谓自下而上分析就是自左至右扫描输入串，自下而上进行分析；通过反复查找当前句型的句柄(最左直接短语)，并使用产生式规则将找到的句柄归约为相应的非终结符。这样逐步进行“归约”，直到归到文法的开始符号；或者说，从语法树的末端开始，步步向上“归约”，直到根结点。



自下而上分析法是一种“移进—归约”法，这是因为在自下而上分析过程采用了一个先进后出的分析栈。分析开始后，把输入符号自左至右逐个移进分析栈，并且边移入边分析，一旦栈顶的符号串形成某个句型的句柄时就进行一次归约，即用相应产生式的左部非终结符替换当前句柄。接下来继续查看栈顶是否形成新的句柄，若为句柄则再进行归约；若栈顶不是句柄则继续向栈中移进后续输入符号。不断重复这一过程，直到将整个输入串处理完毕。若此时分析栈只剩有文法的开始符号则分析成功，即确认输入串是文法的一个句子；否则，即认为分析失败。

(1) $S \rightarrow cAd$

(2) $A \rightarrow ab$

(3) $A \rightarrow a$ 识别输入串 $w=cabd$ 是否为该文法的句子

自上而下的语法分析

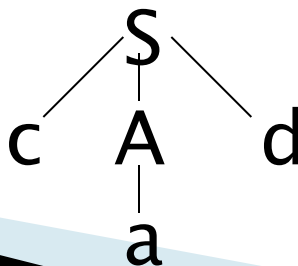
若 $S \Rightarrow cAd$ 后选择(3)扩展A,

$S \Rightarrow cAd \Rightarrow cad$

那么, w 的第二个符号可以与叶子结点a
得以匹配, 但第三个符号却不能与下一
叶子结点d匹配, 宣告分析失败。

这意味着, 识别程序不能为串 w 构造语
法树, 即 $cabd$ 不是句子。这显然是错误
的结论。

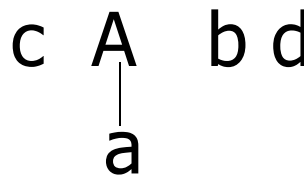
导致失败的原因是在分析中对A的扩展
选择不是正确的。



自下而上的语法分析

对串 $cabd$ 的分析中, 如果不是用
产生式(2)选择 ab , 而是用产生式
(3)选择 a , 将 a 归约到了 A , 那么
最终就达不到归约到 S 的结果,
因而也无从知道 $cabd$ 是一个句子

$c \underline{a} b d$



句型分析的有关问题

- 在自上而下的分析方法中如何选择使用哪个产生式进行推导？

对于符号B有n条规则： $B \rightarrow A1 | A2 | \dots | An$ ，那么用哪个右部替代B？

- 在自下而上的分析方法中如何识别可归约的串？

在分析程序工作的每一步，都是从当前串中选择一个子串，将它归约到某个非终结符号，该子串称为“可归约串”

4.6、自顶向下语法分析方法

包括递归下降分析法和预测分析法。

核心思想：根据下一个输入符号选择产生式，产生式选择不当则引起回溯。

$S \rightarrow AB$

$A \rightarrow aA \mid \varepsilon$

$B \rightarrow b \mid bB$

$a \ a \ a \ b \ \mathbf{b}.$

S

(1) $\Rightarrow A...$

(2) $\Rightarrow aA...$

(3) $\Rightarrow aaA...$

(4) $\Rightarrow aaaA...$

(5) $\Rightarrow aaa \varepsilon B...$

(6) $\Rightarrow aaab$

$S \rightarrow AB$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow \varepsilon$

$B \rightarrow b$

aaabb.

S

(1) $\Rightarrow A...$

(2) $\Rightarrow aA...$

(3) $\Rightarrow aaA...$

(4) $\Rightarrow aaaA...$

(5) $\Rightarrow aaa \varepsilon B$

(6) $\Rightarrow \mathbf{aaa \ b \ B}$

(7) $\Rightarrow aaabb$

$S \rightarrow AB$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow aA$

$A \rightarrow \varepsilon$

$B \rightarrow \mathbf{bB}$

$B \rightarrow b$

4.6.1、引起回溯的原因

若文法中存在下述两种情况，则会给分析带来问题。

〈1〉若存在形如 $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ 的产生式，即A产生式中有多于一个候选项的前缀相同，称为**公共左因子**（或简称左因子），则可能会造成虚假匹配，使得分析过程中可能需要进行大量回溯，从而造成分析效率低、语义动作难以恢复、以及出错位置的报告不确切等。

〈2〉若存在形如 $A \rightarrow A \alpha$ 的产生式，则分析过程中若一旦采用了该产生式去替换，就会陷入死循环而使分析无法进行下去。产生式的这种形式被称为**左递归**。

为了避免上述弱点，需要对文法进行重写：

- ✓ 提取左因子，以避免回溯；
- ✓ 消除左递归，以避免陷入死循环。

为了实现确定的(即无回溯的)自上而下分析, 则要求文法满足下述两个条件:

(1) 文法不含左递归, 即不存在这样的非终结符号 A : 有 $A \rightarrow A\dots$ 存在或者有 $AA \alpha$;

(2) 无回溯, 对文法的任一非终结符号, 当其产生式右部有多个候选式可供选择时, 各候选式所推出的终结符号串的首字符集合要两两不相交。

自上而下的语法分析——左递归规则

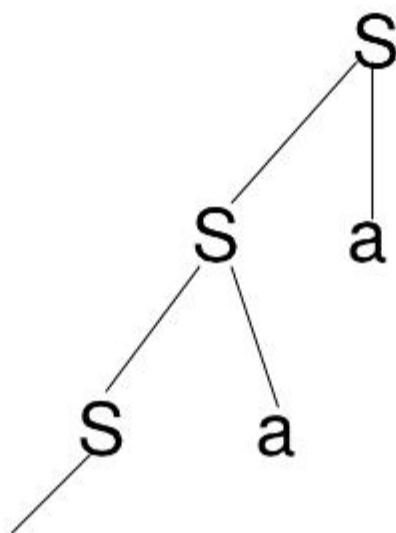
$G[S]:$ $S \rightarrow Sa$

$S \rightarrow b$

$L = \{ba^n \mid n \geq 1\}$

$W = baaa$

S
 b



消除左递归

若文法 G 中的非终结符 A 对某个符号序列 α 存在推导 $A \xRightarrow{+} A\alpha$ ，则称 G 是左递归的。若 G 中有形如 $A \rightarrow A\alpha$ 的产生式，则称该产生式对 A 直接左递归。例：非直接左递归的文法

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Ac \mid Sd \mid \varepsilon$$

消除左递归

+

消除文法的直接左递归

1) 首先考虑仅有两个候选项的直接左递归的产生式

$A \rightarrow A\alpha \mid \beta$ ，它仅有两个候选项，其中一个是直接左递归的，另一个不含直接左递归。

2) 可以看出产生式 $A \rightarrow A\alpha \mid \beta$ 产生的语言集与 $\beta\alpha^*$ 产生的语言集等价，该语言集为 β 后边跟随若干个 α （可以是0个）的字符串集合。

3) 由此可以把 $A \rightarrow \beta\alpha^*$ 用非左递归的 $A \rightarrow \beta A'$ 和

$A' \rightarrow \alpha A' \mid \epsilon$ 取代。

消除左递归

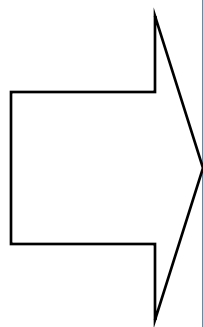
+

更一般的形式: $A \rightarrow A \alpha_1 \mid A \alpha_2 \mid \dots \mid A \alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$



$$\begin{array}{lcl} A & \rightarrow & \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A' \\ A' & \rightarrow & \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon \end{array}$$

例：表达式文法直接左递归的消除

$$\begin{aligned} E &\rightarrow E + T \mid T \\ T &\rightarrow T * F \mid F \\ F &\rightarrow (E) \mid id \end{aligned}$$

$$\begin{aligned} E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid \varepsilon \\ F &\rightarrow (E) \mid id \end{aligned}$$

练习：消除文法 $G[S]: S \rightarrow (L) \mid a \quad L \rightarrow L, S \mid S$ 的直接左递归

消除非直接左递归算法

将非终结符排序： $A_1 A_2 \dots A_n$ ，使得它们在产生式序列中按序出现

for i in $2..n$

 loop for j in $1..i-1$ // j 总是小于 i

 loop

 用 $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ 的右部替换每个形如 $A_i \rightarrow A_j \gamma$ 产生式中的 A_j ，得到新产生式： $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$;

 消除 A_i 产生式中的直接左递归;

 end loop;

 end loop;

该算法并不能消除所有文法中的左递归，若文法 G 产生句子的过程中会出现 $A \Rightarrow^+ A$ 的推导，则无法消除左递归。

如何消除一个文法的一切左递归呢？如果一个文法不含回路(形如 AA 的推导)，且产生式的右部也不含 ε 的候选式，那么，下述算法将消除文法的左递归：

(1) 将文法 $G[S]$ 的所有非终结符按一给定的顺序排列：
 $A_1、A_2、\dots、A_n$ ；

(2) 执行下述循环语句将间接左递归改为直接左递归：

for ($i=1; i \leq n; i++$)

for ($j=1; j \leq i-1; j++$)

{把一个形如：

$$\begin{cases} A_i \rightarrow A_j \gamma \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_k \\ A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k \end{cases}$$

的产生式改写为：

$$A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n ;$$

按消除直接左递归的方法消除 A_i 的直接左递归；

}

(3) 化简由(2)所得的文法，即去掉那些从开始符号 S 出发，在推导中无法出现的非终结符的产生式(去掉多余产生式)。

消除下面文法中的左递归:

$R \rightarrow Sa|a$

$Q \rightarrow Rb|b$

$S \rightarrow Qc|c$



例如，将文法(3.3)的非终结符排序为R、Q、S。对R来说，它不存在直接左递归；当把R代入到Q的有关候选后得到改变后的Q产生式为

$$Q \rightarrow Sab \mid ab \mid b$$

现在的Q同样不含直接左递归，再把它代入到S的有关候选后得到改变的S产生式为

$$S \rightarrow Sabc \mid abc \mid bc \mid c$$


经过消除了S的直接左递归后，即得到了整个文法G'[S]为：

$$G'[S]: S \rightarrow abcS' \mid bcS' \mid cS'$$

$$S' \rightarrow abcS' \mid \varepsilon$$

$$Q \rightarrow Sab \mid ab \mid b$$

$$R \rightarrow Sa \mid a$$

显然，关于Q和R的产生式已为多余，因此化简后的最终文法G''[S]为：

$$G''[S]: S \rightarrow abc S' \mid bc S' \mid c S'$$

$$S' \rightarrow abc S' \mid \varepsilon \quad (3.4)$$

实际上，我们也可以用数学中的分配律来消除文法中的左递归。对文法(3.3)，首先将R的产生式代入到Q的产生式中并按分配律展开(注：“(”和“)”为元语言符号)得

$Q \rightarrow (Sa \mid a)b \mid b$ 展开后：

$Q \rightarrow Sab \mid ab \mid b$

再将改变后Q的产生式代入到S的产生式中并按分配律展开得

$S \rightarrow (Sab \mid ab \mid b)c \mid c$ 展开后：

$S \rightarrow Sabc \mid abc \mid bc \mid c$

在消除S的直接左递归后同样得到文法(3.4)。

消除文法G[S]: $S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$ 的左递归

① S和A排序, S在先 A在后;

② 用S右部替换 $A \rightarrow Sd$ 中的S, 得到新产生式 $A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$

消除文法G[S]: $S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$ 的左递归

① S和A排序，S在先 A在后；

② 用S右部替换 $A \rightarrow Sd$ 中的S，得到新产生式 $A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$

消除新产生式中的直接左递归，最后得到等价的文法如下：

$S \rightarrow Aa \mid b$

$A \rightarrow bdA' \mid A'$

$A' \rightarrow cA' \mid adA' \mid \varepsilon$

消除文法G[S]: $S \rightarrow Aa \mid b$

$A \rightarrow Ac \mid Sd \mid \varepsilon$ 的左递归

① S和A排序，S在先 A在后；

② 用S右部替换 $A \rightarrow Sd$ 中的S，得到新产生式 $A \rightarrow Ac \mid Aad \mid bd \mid \varepsilon$

消除新产生式中的直接左递归，最后得到等价的文法如下：

$S \rightarrow Aa \mid b$

$A \rightarrow bdA' \mid A'$

$A' \rightarrow cA' \mid adA' \mid \varepsilon$

练习：消除文法G[S]: $S \rightarrow Ra \mid a$ $R \rightarrow Sb \mid b$ 的左递归

4. 消除回溯

回溯发生的原因在于候选式存在公共的左因子，如产生式A如下：

$$A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$$

此时，如果输入串待分析的字符串前缀为 α ，则选用哪个候选式以寻求与输入串匹配就难以确定。倘若候选式不含公共左因子，则推导出的首字符能与输入串匹配的那个候选式便是惟一的匹配。在文法 $G[S]$ 中的每个非终结符相应的产生式右部其候选式均不含公共左因子的情况下，语法分析的匹配过程都是惟一匹配，无需试探；这时若匹配失败，则意味着输入串不是句子。

提取左因子

对于有左因子的文法，推导过程中会出现无法确定用A产生式的哪个候选项替换A。这时可以重写A产生式来推迟这种决定，直到看见足够的输入，能正确决定所需选择为止。这一过程称为提取左因子。

例如：悬空else文法

$$S \rightarrow iCtS \mid iCtSeS \mid a$$
$$C \rightarrow b$$

提取文法的左因子的方法

1) 为每个产生式A，找出其候选项中最长公共前缀 α ，重排A 产生式如下，其中 γ 是不以 α 为前缀的其他候选项。

$$A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma$$

2) 用下述产生式替代之。

$$A \rightarrow \alpha A' \mid \gamma$$

$$A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$$

3) 重复此过程，直到所有A产生式的候选项中均不再有公共前缀。

上例经提取左因子后变为：

$$\begin{array}{lcl} S & \rightarrow & iCtSS' \mid a \\ S' & \rightarrow & eS \mid \varepsilon \\ C & \rightarrow & b \end{array}$$

我们也可以用数学中提取公共因子的办法来提取公共左因子。如对式(3.5)提取公共(左)因子后得

$$A \rightarrow \delta (\beta_1 \mid \beta_2 \mid \dots \mid \beta_i) \mid \beta_{i+1} \mid \dots \mid \beta_j$$

(注：“(”与“)”为元语言符号)

将产生式中由“(”和“)”括起的部分以非终结符A'命名则得到式(3.6)。

5. 递归下降分析器

在不含左递归和每个非终结符的所有候选终结首符集都两两不相交的条件下，我们就可能构造一个不带回溯的自上而下的分析程序，这个分析程序是由一组递归过程(或函数)组成的，每个过程(或函数)对应文法的一个非终结符。这样的一个分析程序称为递归下降分析器。



4.6.2、递归下降分析

递归下降分析是**以程序的方式**直接模拟**产生式产生语言的过程**，它的基本思想是：

1) 为每一个**非终结符**构造一个子程序，每一个子程序的过程体中按该产生式的候选项展开，遇到终结符直接匹配，而遇到非终结符就调用相应非终结符的子程序。

例如：为产生式 $Z \rightarrow aBa$ 构造的递归子程序如下：

```
procedure Z;  
Begin  
    Match(a); B; Match(a);  
End;
```

4.6.2、递归下降分析

2) 分析从调用文法开始符号的子程序开始，直到所有**非终结符都展开为终结符**并得到匹配为止，若分析过程中达到这一步则分析成功，否则输入中有语法错误。

3) 由于文法是递归定义的，因此子程序也是递归的，被称为递归下降子程序。递归下降分析限制文法**不能有公共左因子和左递归**。

4) 对于规模比较小的语言，递归下降子程序方法是很有效的方法，它**简单灵活**，容易构造，缺点是**程序与文法直接相关**，对文法的任何改变均需对程序进行相应的修改。



递归子程序实现

在文法表示中引入几个符号（扩充BNF范式）

- ① { }：{ }中的内容可以重复0或若干次。
- ② []：[]中的内容是可选的。
- ③ ()：利用()可以改变运算的优先级和结合性。

$$\begin{aligned} L &\rightarrow E ; L \mid \varepsilon \\ E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid T \bmod F \mid F \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$

消除左递归

$$\begin{aligned} L &\rightarrow E ; L \mid \varepsilon \\ E &\rightarrow T E' \\ E' &\rightarrow + T E' \mid - T E' \mid \varepsilon \\ T &\rightarrow F T' \\ T' &\rightarrow * F T' \mid / F T' \mid \bmod F T' \mid \varepsilon \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$
$$\begin{aligned} L &\rightarrow \{ E ; \} \\ E &\rightarrow T \{ (+|-) T \} \\ T &\rightarrow F \{ (*|/|\bmod) F \} \\ F &\rightarrow (E) \mid \text{id} \mid \text{num} \end{aligned}$$

例:表达式文法的递归下降语法分析

$L \rightarrow \{ E; \}$

$E \rightarrow T \{ (+|-) T \}$

$T \rightarrow F \{ (*|/|\text{mod}) F \}$

$F \rightarrow (E) | \text{id} | \text{num}$

$\langle \text{表达式组} \rangle \rightarrow \{ \langle \text{表达式} \rangle ; \}$

$\langle \text{表达式} \rangle \rightarrow \langle \text{项} \rangle \{ (+|-) \langle \text{项} \rangle \}$

$\langle \text{项} \rangle \rightarrow \langle \text{因子} \rangle \{ (*|/) \langle \text{因子} \rangle \}$

$\langle \text{因子} \rangle \rightarrow ' (' \langle \text{表达式} \rangle ') ' \quad | \quad \text{id} \quad |$

procedure expr;
begin
 term;
 while ch in [plus, minus] do
 begin
 getch(ch); term;
 end
 end;
end;

procedure term;
begin
 factor;
 while ch in [times, slash] do
 begin
 getch(ch); factor;
 end
 end;
end;

经过消去直接左递归后得到文法 $G'[E]$ 为:

$G'[E]: E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT \mid \varepsilon$

$F \rightarrow (E) \mid i \quad (3.2)$

例如，文法(3.2)对应的递归下降分析器如下：

$G'[E]: E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT \mid \varepsilon$

$F \rightarrow (E) \mid i$

```
void match (token t)
```

```
{
```

```
    if (lookahead == t)
```

```
        lookahead = nexttoken;
```

```
    else error ( );
```

```
}
```

```
void E( )
```

```
{
```

```
    T( );
```

```
    E'( );
```

}

void E'()

G'[E]: $E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT \mid \varepsilon$

$F \rightarrow (E) \mid i$

{

if (lookahead == '+')

{

match ('+');

T();

E'();

}



}

$G'[E]: E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT \mid \varepsilon$

$F \rightarrow (E) \mid i$

```
void T( )
```

```
{
```

```
    F( );
```

```
    T'( );
```

```
}
```

```
void T'( )
```

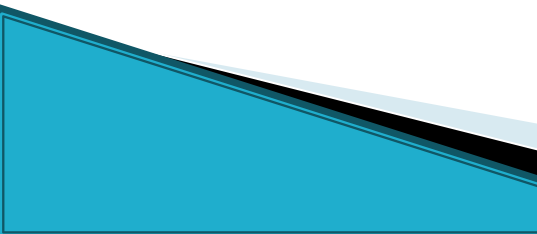
```
{
```

```
    if (lookahead == '*')
```

```
    {
```

```
        match ('*');
```

```
        F( );
```



T'();

}

}

G'[E]: E → TE'

E' → +TE' | ε

T → FT'

T' → *FT | ε

F → (E) | i

void F()

{

if (lookahead == 'i')

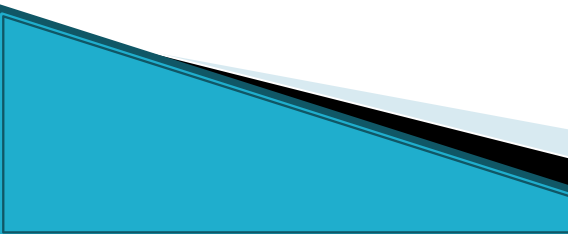
match ('i');

else if (lookahead == '(')

{

match '(';)

E ();



$G'[E]: E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT \mid \varepsilon$

$F \rightarrow (E) \mid i$

if (lookahead == ')')

match (')');

else error ();

}

else error ();

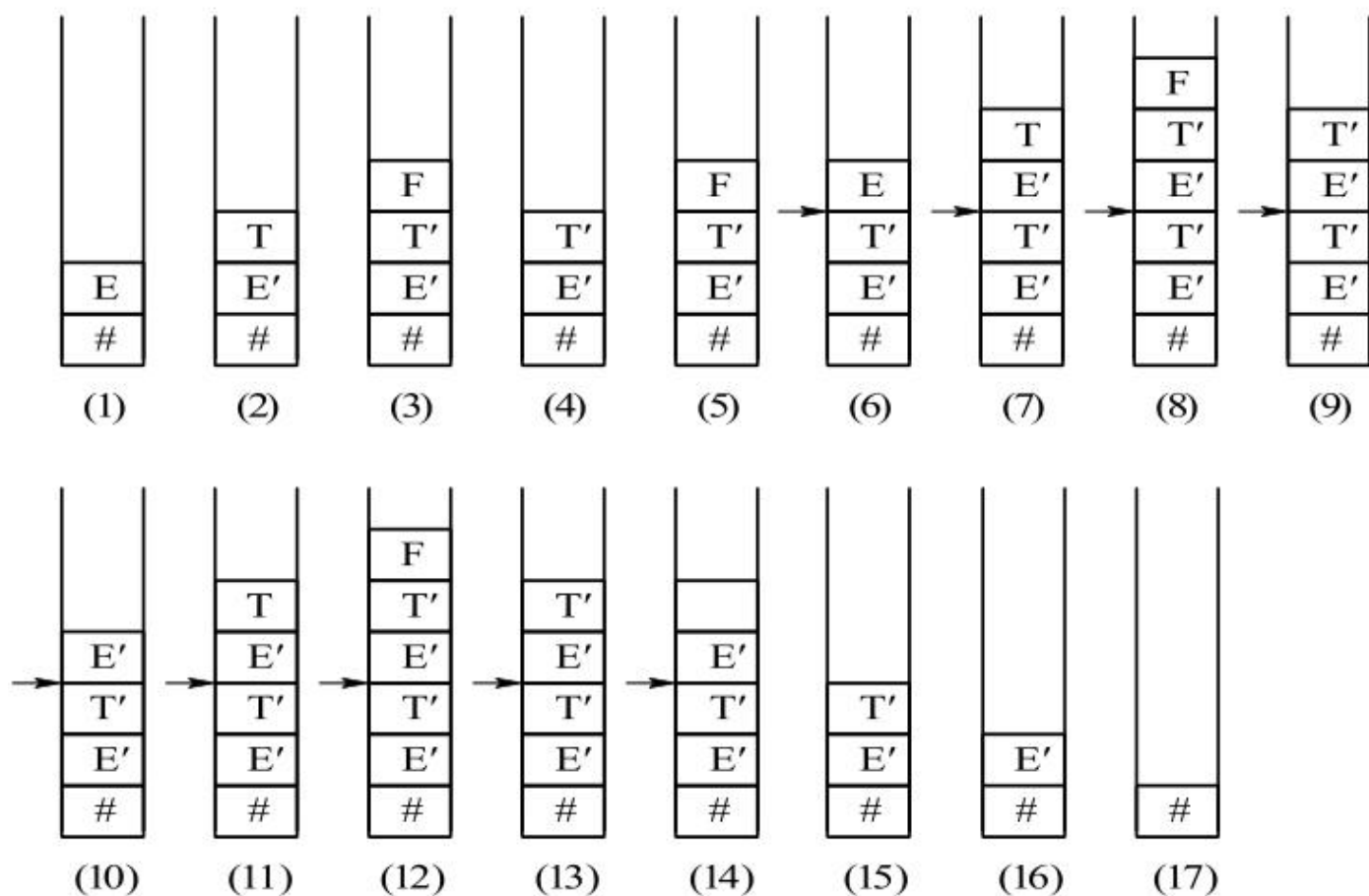
}

我们知道，关于 E' 的产生式是

$E' \rightarrow +TE' \mid \varepsilon$

即 E' 只有两个候选；第一个候选的开头终结符为 $+$ ，第二个候选为 ε 。这就是说，当 E' 面临输入符号“ $+$ ”时就令第一个候选进入工作，而当面临任何其它符号时， E' 就自动认为获得了匹配。递归函数 E' 就是根据这一原则设计的。

例如，我们将递归函数的调用以栈的形式模拟来分析输入串 $\# i_1*(i_2+i_3)\#$ 的语法分析过程；在此，“ $\#$ ”为输入串 $i_1*(i_2+i_3)$ 的分隔符。进行语法分析时，首先将“ $\#$ ”和文法开始符 E 压入栈中，当语法分析进行到栈中仅剩“ $\#$ ”而输入串扫描指针已指向输入串尾部的“ $\#$ ”时，则语法分析成功，分析过程如图3-12所示。



说明:

(1)~(3): $\#i_1(i_2+i_3)\#$

(4): $\#i_1^*(i_2+i_3)\#$

(5): $\#i_1^*(i_2+i_3)\#$

(6)~(8): $\#i_1^*(i_2+i_3)\#$

(9)~(10): $\#i_1(i_2+i_3)\#$

(11)~(12): $\#i_1(i_2+i_3)\#$

(13)~(14): $\#i_1(i_2+i_3)\#$

(15)~(17): $\#i_1(i_2+i_3)\#$

图3-12 输入串 $i_1*(i_2+i_3)$ 的语法分析